Technical Architecture – Siffrum Platform

1. Technology Stack

The proposed system will be built using a modern full■stack architecture that supports scalability, security, and AI integration.

Frontend technologies:
• HTML5
• CSS3
• JavaScript
• React.js / Next.js

Key functions include loan dashboards, user data input forms, financial analytics graphs, and repayment strategy visualization.

2. Programming Language

Frontend: JavaScript

Backend: Python

Python is selected because of its strong ecosystem for machine learning, fast API development capability, and robust fintech/data■analysis libraries.

Framework options include FastAPI and Django REST Framework.

3. Backend Workflow

The backend follows a client–server model where the frontend communicates with backend services using secure APIs.

Workflow:
1. User logs into the platform
2. User enters loan information
3. Frontend sends API request
4. Backend processes and validates data
5. Data is stored in database
6. Machine learning models analyze repayment risk
7. System generates recommendations and stress scores
8. Results are returned to the user dashboard

4. API Structure

Authentication APIs
POST /api/register
POST /api/login
GET /api/user-profile

Loan Management APIs
POST /api/add-loan
GET /api/get-loans
PUT /api/update-loan
DELETE /api/delete-loan

Analytics APIs
GET /api/stress-score
GET /api/loan-summary
GET /api/repayment-plan

Simulation APIs
POST /api/simulate-snowball
POST /api/simulate-avalanche
POST /api/simulate-refinance

5. Database Design

Recommended Databases:
• PostgreSQL for structured financial data
• MongoDB for behavioral logs and analytics

Core tables:

Users Table
user_id, name, email, password_hash, monthly_income, employment_type, created_at

Loans Table
loan_id, user_id, principal_amount, interest_rate, tenure_months, emi_amount, start_date, loan_status

Transactions Table
transaction_id, user_id, loan_id, transaction_amount, category, date

Stress Analysis Table
analysis_id, user_id, stress_score, default_probability, recommended_strategy, created_at

6. System Architecture Overview

The architecture consists of five major layers:
• User Interface Layer
• API Gateway Layer
• Application Logic Layer
• Machine Learning Engine
• Data Storage Layer

This layered architecture ensures modular development, scalability, and secure data processing.

System Architecture Diagram

```
               ■■■■■■■■■■■■■■■■■■■■■■■
               ■        User         ■
               ■   (Mobile / Web)   ■
               ■■■■■■■■■■■■■■■■■■■■■■
                         ■
               ■■■■■■■■■■■▼■■■■■■■■■■■
               ■   Frontend Layer   ■
               ■ React / HTML / JS  ■
               ■ Dashboards & UI    ■
               ■■■■■■■■■■■■■■■■■■■■■■
                         ■ API
               ■■■■■■■■■■■▼■■■■■■■■■■■
               ■      API Layer     ■
               ■ FastAPI / Django   ■
               ■■■■■■■■■■■■■■■■■■■■■■
                         ■
               ■■■■■■■■■■■▼■■■■■■■■■■■
               ■ Application Logic  ■
               ■ Loan Processing    ■
               ■ Stress Calculation ■
               ■■■■■■■■■■■■■■■■■■■■■■
                         ■
           ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
           ■                            ■
■■■■■■■■■■■■■▼■■■■■■■■■■■■      ■■■■■■■■■■■■■▼■■■■■■■■■■■■
■    AI / ML Engine     ■      ■   Simulation Engine ■
■ Default Prediction   ■      ■ Snowball Strategy   ■
■ Stress Forecasting   ■      ■ Avalanche Strategy  ■
■■■■■■■■■■■■■■■■■■■■■■■■      ■■■■■■■■■■■■■■■■■■■■■■■■
           ■                            ■
           ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
                         ■
               ■■■■■■■■■■■▼■■■■■■■■■■■
               ■    Database Layer  ■
               ■ PostgreSQL/MongoDB ■
               ■■■■■■■■■■■■■■■■■■■■■■
                         ■
               ■■■■■■■■■■■▼■■■■■■■■■■■
               ■   Cloud Platform   ■
               ■ AWS Infrastructure ■
               ■■■■■■■■■■■■■■■■■■■■■■
```